Árboles de decisión y métodos de ensamblaje

Tomas Santana
October, 2026

Índice

1. Parte 1: Árboles de decisión 1
   1.1. Un modelo no lineal: el ejemplo del esquí 1
   1.2. Particionamiento recursivo 2
   1.3. Qué predice cada hoja 3
2. Parte 2: ¿Qué pregunta hacer? 4
   2.1. Ganancia de una división 4
   2.2. Primer intento: el error de clasificación 4
   2.3. La impureza de Gini 5
   2.4. Por qué Gini funciona: concavidad 5
   2.5. Entropía cruzada 6
   2.6. Cuántas preguntas hay que probar 7
3. Parte 3: Regresión, variables categóricas y regularización 7
   3.1. Árboles de regresión 7
   3.2. Variables categóricas 9
   3.3. Regularización y poda 9
   3.4. Evaluar un clasificador: exactitud y matriz de confusión 10
   3.5. Ventajas y desventajas 11
4. Parte 4: Ensamblaje: bagging y random forests 12
   4.1. Por qué promediar modelos 12
   4.2. Bootstrap 13
   4.3. Bagging 14
   4.4. Random forests 14
5. Parte 5: Boosting 15
   5.1. Idea: combinar modelos débiles 15
   5.2. AdaBoost 15
   5.3. Boosting con pérdida cuadrática: ajustar los residuos 17
   5.4. Gradient boosting 19
   5.5. XGBoost 20
6. Resumen 21

1. Parte 1: Árboles de decisión

1.1. Un modelo no lineal: el ejemplo del esquí

Supongamos que queremos predecir si se puede esquiar en un lugar y un momento dados, a partir de dos features: el mes del año (de 1 a 12) y la latitud del lugar (de $-90°$ en el Polo Sur a $90°$ en el Polo Norte, con $0°$ en el Ecuador). En el hemisferio norte se puede esquiar al principio y al final del año. En el hemisferio sur, a mitad de año. Cerca del Ecuador, nunca.

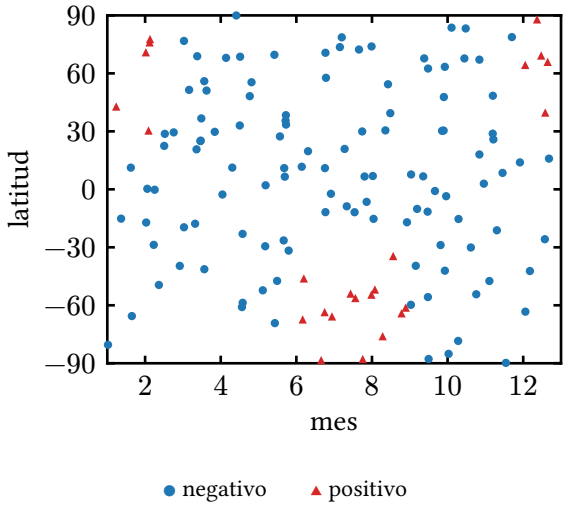


Figura 1: 140 ejemplos del problema del esquí. Los positivos (se puede esquiar) aparecen en tres zonas: arriba a la izquierda y arriba a la derecha (invierno del hemisferio norte), y abajo al centro (invierno del hemisferio sur). Ninguna recta separa las dos clases.

La Figura 1 muestra los datos. Un clasificador lineal no puede separarlos: los positivos están en tres zonas separadas. Podríamos inventar features nuevas, como en las clases anteriores, pero hay una forma mucho más directa de describir esta frontera: con preguntas. ¿Estamos al norte de los 30°? Si es así, ¿estamos antes de marzo? Es el mismo procedimiento del juego de las 20 preguntas.

1.2. Particionamiento recursivo

Un árbol de decisión divide el espacio de las features en regiones haciendo una pregunta a la vez. Cada pregunta mira una sola feature y la compara con un umbral.

Formalmente, empezamos con una región R_p (la región "padre"), que al principio es todo el espacio. Escogemos una feature j y un umbral t, y dividimos R_p en dos regiones hijas:

$$R_1 = \{\mathbf{x} \in R_p : x_j < t\}, \qquad R_2 = \{\mathbf{x} \in R_p : x_j \geq t\}.$$

Luego repetimos el procedimiento por separado en R_1 y en R_2, y así sucesivamente. Por eso se dice que el algoritmo es recursivo y de arriba hacia abajo (top-down): empieza con todo el espacio y lo va partiendo en pedazos cada vez más pequeños. Además es codicioso (greedy): en cada paso escoge la mejor pregunta en ese momento, sin pensar en las preguntas que vendrán después.

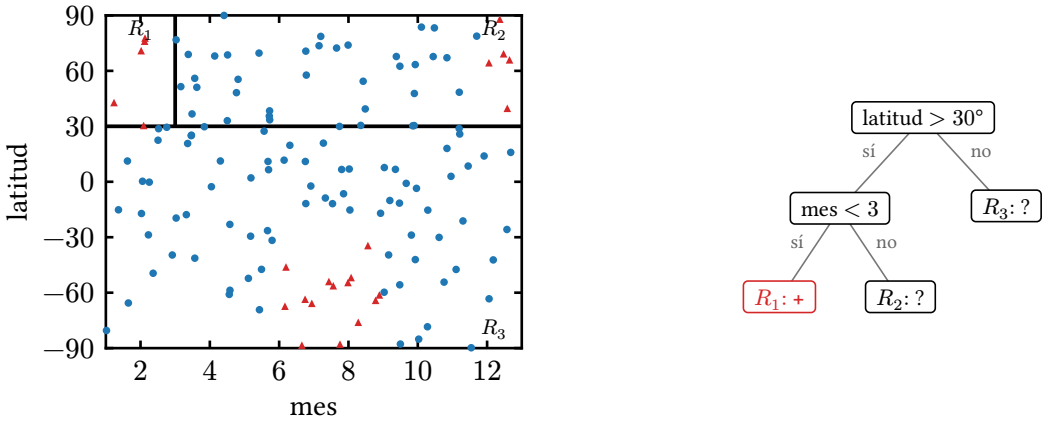


Figura 2: Las dos primeras preguntas. La primera, latitud $> 30°$, divide el plano con una recta horizontal. La segunda, mes < 3, divide solo la región de arriba, con un segmento vertical. Las regiones que quedan corresponden a las hojas del árbol de la derecha. La región de arriba a la izquierda contiene 5 positivos y 0 negativos: ya es pura. En las otras dos regiones todavía hay ejemplos de las dos clases, marcadas con "?".

Si seguimos haciendo preguntas, eventualmente todas las regiones quedan puras. La Figura 3 muestra un árbol que clasifica correctamente todos los ejemplos.

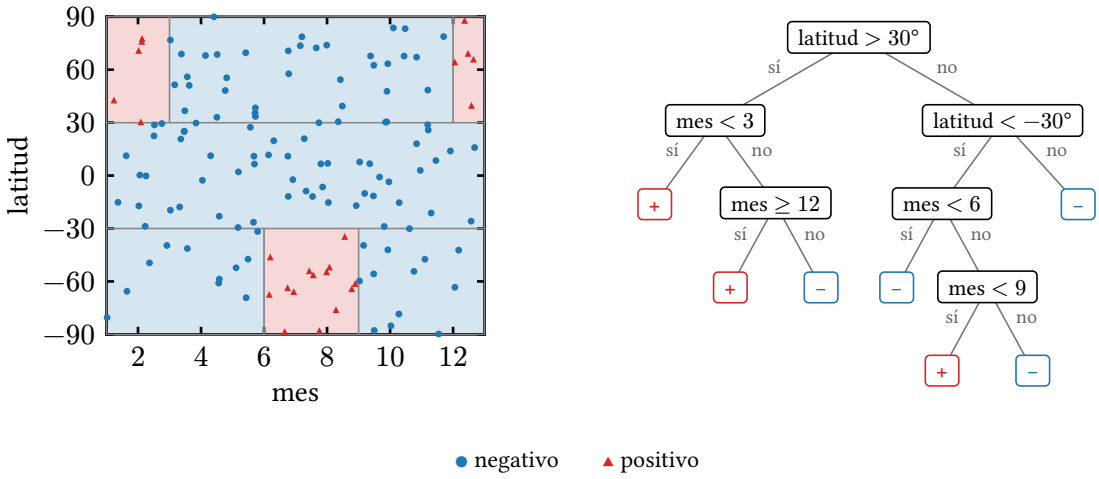


Figura 3: Un árbol que clasifica correctamente todos los ejemplos. Tiene 7 hojas, y cada hoja corresponde a una de las 7 regiones rectangulares de la izquierda. Seguimos la rama "sí" si la respuesta a la pregunta es afirmativa.

Algunas observaciones:

- Cada pregunta compara una sola feature con un umbral, así que las fronteras son siempre paralelas a los ejes y las regiones son rectángulos (o, con d features, cajas en $\mathbb{R}^d$).
- Las regiones no se solapan y cubren todo el espacio: cada punto cae en exactamente una región.
- Cada región corresponde a una hoja del árbol (un nodo sin hijos). El número de regiones es el número de hojas.
- Una misma feature se puede usar varias veces, con distintos umbrales, en distintas ramas.

1.3. Qué predice cada hoja

Supongamos que el árbol tiene J hojas, con regiones $R_1, \ldots, R_J$. El árbol asigna a cada región una predicción w_j, y para un punto $\mathbf{x}$ predice el valor de la región donde cae:

$$f(\mathbf{x}) = \sum_{j=1}^{J} w_j \mathbb{1}\{\mathbf{x} \in R_j\}.$$

Todos los términos de la suma son cero excepto el de la región que contiene a $\mathbf{x}$. En clasificación, w_j es la clase más frecuente entre los ejemplos de entrenamiento que caen en R_j (un voto de mayoría). En regresión, como veremos en la Parte 3, w_j es el promedio de sus etiquetas.

Falta la pregunta más importante: ¿cómo escoger la pregunta (la feature j y el umbral t) en cada paso?

2. Parte 2: ¿Qué pregunta hacer?

2.1. Ganancia de una división

Para comparar preguntas necesitamos medir qué tan "mezclada" está una región. Llamamos $L(R)$ a esa medida, la impureza de la región, y la definiremos en un momento. Una buena pregunta es la que deja hijos mucho menos mezclados que el padre.

Para una región R, sea $\hat{p}_c$ la proporción de ejemplos de entrenamiento de la clase c en R. La impureza $L(R)$ será una función de estas proporciones. Como las regiones hijas pueden tener tamaños muy distintos, promediamos sus impurezas ponderando por el número de ejemplos que contiene cada una. La ganancia de dividir R_p en R_1 y R_2 es

$$\text{ganancia} = L(R_p) - \left(\frac{|R_1|}{|R_p|} L(R_1) + \frac{|R_2|}{|R_p|} L(R_2) \right),$$

donde $|R|$ es el número de ejemplos de entrenamiento en R. En cada paso escogemos la feature j y el umbral t que maximizan la ganancia. Como $L(R_p)$ no depende de la pregunta, esto es lo mismo que minimizar la impureza promedio de los hijos.

2.2. Primer intento: el error de clasificación

La medida más obvia es la fracción de ejemplos que clasificaríamos mal si predijéramos la clase mayoritaria de la región:

$$L_{\text{error}}(R) = 1 - \max_c \hat{p}_c.$$

Parece razonable, pero tiene un problema. Consideremos una región padre con 900 positivos y 100 negativos, y dos posibles divisiones (Figura 4).



Figura 4: Dos formas de dividir una región con 900 positivos y 100 negativos. En ambas, uno de los hijos es puro (solo tiene positivos), pero la división de la derecha aísla 500 positivos en lugar de 200.

Intuitivamente, la división B es mejor: deja 500 positivos en una región pura, contra 200 de la división A. Pero el error de clasificación no ve ninguna diferencia. En el padre predecimos positivo y nos equivocamos en 100 de 1000 ejemplos. En la división A, los dos hijos predicen positivo y siguen equivocándose en los mismos 100 negativos: $\frac{800}{1000} \cdot \frac{100}{800} + \frac{200}{1000} \cdot 0 = 0.1$. En la división B ocurre lo mismo: $\frac{500}{1000} \cdot \frac{100}{500} + 0 = 0.1$. La ganancia es cero en los dos casos, como si ninguna de las dos preguntas sirviera para nada.

El problema es que el error de clasificación solo cambia si alguna región cambia de clase mayoritaria. Muchas preguntas buenas, que separan bien los datos sin cambiar la clase mayoritaria, quedan con ganancia cero. Necesitamos una medida más sensible.

2.3. La impureza de Gini

Imaginemos el siguiente juego. Sacamos al azar un ejemplo de la región R, y en lugar de predecir siempre la clase mayoritaria, le asignamos una etiqueta al azar, sorteada con las mismas proporciones de la región: la clase c con probabilidad $\hat{p}_c$. ¿Con qué probabilidad nos equivocamos?

El ejemplo es de la clase c con probabilidad $\hat{p}_c$, y en ese caso nos equivocamos si la etiqueta sorteada es otra, lo que ocurre con probabilidad $1 - \hat{p}_c$. Sumando sobre las clases:

$$L_{\text{Gini}}(R) = \sum_c \hat{p}_c (1 - \hat{p}_c) = 1 - \sum_c \hat{p}_c^2.$$

(La segunda forma sale de que $\sum_c \hat{p}_c = 1$.) A esto se le llama la impureza de Gini. Otra forma equivalente de verla: es la probabilidad de que dos ejemplos sacados al azar de la región (con reemplazo) sean de clases distintas.

- Si la región es pura, todos los ejemplos son de la misma clase y nunca nos equivocamos: $L_{\text{Gini}} = 0$.
- Si la región está muy mezclada, nos equivocamos mucho. Con dos clases, el máximo es $\frac{1}{2}$, cuando $\hat{p} = \frac{1}{2}$.

En clasificación binaria, si $\hat{p}$ es la proporción de positivos, la fórmula queda

$$L_{\text{Gini}}(R) = \hat{p}(1 - \hat{p}) + (1 - \hat{p})\hat{p} = 2\hat{p}(1 - \hat{p}).$$

Volvamos al ejemplo. El padre tiene $\hat{p} = 0.9$, así que su impureza es $2 \cdot 0.9 \cdot 0.1 = 0.18$. En la división A, el hijo $(700+, 100-)$ tiene $\hat{p} = 0.875$ e impureza 0.2188, y el hijo puro tiene impureza 0. El promedio ponderado es $0.8 \cdot 0.2188 + 0.2 \cdot 0 = 0.175$. En la división B, el hijo $(400+, 100-)$ tiene $\hat{p} = 0.8$ e impureza 0.32, y el promedio es $0.5 \cdot 0.32 = 0.16$.

Medida	Padre	Ganancia de A	Ganancia de B
Error de clasificación	0.1	0	0
Gini	0.18	0.005	0.02
Entropía cruzada (bits)	0.469	0.034	0.108

Tabla 1: Ganancia de las dos divisiones de la Figura 4 con tres medidas de impureza. El error de clasificación no distingue las dos divisiones. Gini y la entropía cruzada prefieren la división B.

Con Gini, la ganancia de la división B (0.02) es cuatro veces la de la división A (0.005). Gini sí premia la pregunta que aísla más positivos.

2.4. Por qué Gini funciona: concavidad

Hay una explicación geométrica de por qué Gini ve la diferencia y el error de clasificación no. Pensemos en clasificación binaria y escribamos la impureza como función de la proporción de positivos, $L(\hat{p})$.

Primero, la proporción de positivos del padre es el promedio ponderado de las proporciones de los hijos:

$$\hat{p}_p = \frac{|R_1|}{|R_p|}\hat{p}_1 + \frac{|R_2|}{|R_p|}\hat{p}_2,$$

porque los positivos del padre son los positivos de R_1 más los de R_2. En la división B, por ejemplo, $0.9 = 0.5 \cdot 0.8 + 0.5 \cdot 1$.

Ahora dibujemos la curva $L(\hat{p})$ y marquemos los puntos de los dos hijos de una división, $(\hat{p}_1, L(\hat{p}_1))$ y $(\hat{p}_2, L(\hat{p}_2))$ (Figura 5). La impureza promedio de los hijos es un punto sobre el segmento que los une, justo encima de $\hat{p}_p$. La impureza del padre es el punto de la curva encima de $\hat{p}_p$. La ganancia es la distancia vertical entre los dos.

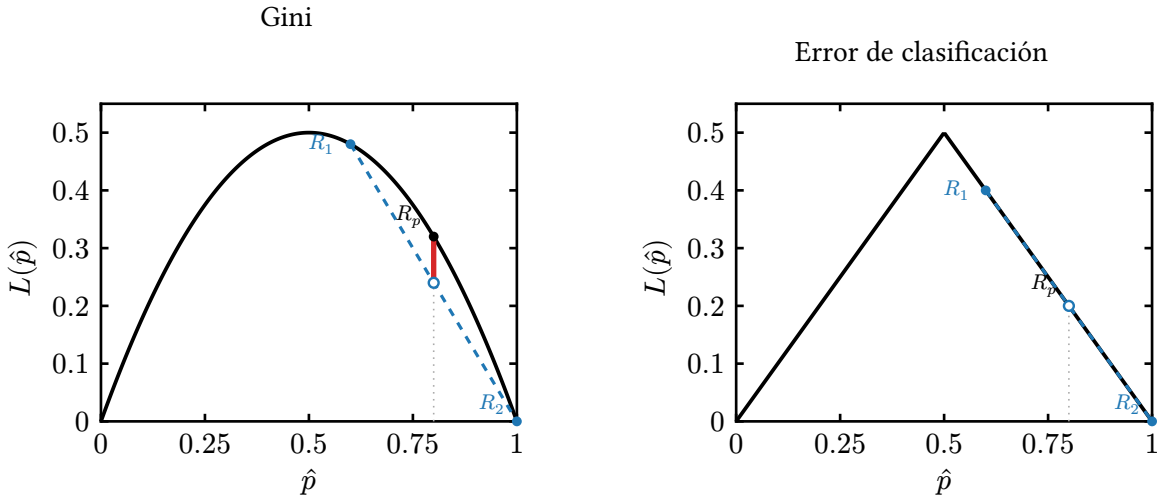


Figura 5: Una división en dos hijos del mismo tamaño, uno con 60% de positivos y otro puro ($\hat{p}_1 = 0.6$, $\hat{p}_2 = 1$), así que el padre tiene $\hat{p}_p = 0.8$. Los puntos azules son los hijos, el círculo blanco es su impureza promedio y el punto negro es el padre. La ganancia es la distancia vertical entre los dos (en rojo). A la izquierda, con Gini, la curva está estrictamente por encima del segmento y la ganancia es positiva (0.08). A la derecha, con el error de clasificación, la curva es una recta a cada lado de $\frac{1}{2}$: como los dos hijos quedan del mismo lado, el segmento coincide con la curva y la ganancia es cero.

La curva de Gini es estrictamente cóncava (tiene forma de domo): cualquier segmento entre dos puntos distintos de la curva queda estrictamente por debajo de ella. Por lo tanto, cualquier división que cambie las proporciones, es decir, con $\hat{p}_1 \neq \hat{p}_2$, tiene ganancia positiva. Y cuanto más separados quedan $\hat{p}_1$ y $\hat{p}_2$, mayor es la ganancia.

La curva del error de clasificación, en cambio, es una pirámide formada por dos rectas. Si los dos hijos quedan del mismo lado de $\frac{1}{2}$ (los dos con mayoría positiva, por ejemplo), el segmento entre ellos queda exactamente sobre la recta y la ganancia es cero. Eso es lo que pasó en las dos divisiones del ejemplo.

2.5. Entropía cruzada

La otra medida de impureza muy usada es la entropía cruzada (o simplemente entropía), que viene de la teoría de la información:

$$L_{\text{entropía}}(R) = -\sum_c \hat{p}_c \log_2 \hat{p}_c.$$

Se interpreta como el número de bits que hacen falta, en promedio, para comunicar la clase de un ejemplo de la región si conocemos las proporciones $\hat{p}_c$. Si la región es pura no hay nada que comunicar, y la entropía es 0. Si hay dos clases mezcladas por igual, hace falta 1 bit.

Como muestra la Figura 6, la entropía y Gini tienen prácticamente la misma forma: las dos son estrictamente cóncavas, valen cero en las regiones puras y son máximas cuando las clases están balanceadas. En la práctica casi siempre escogen las mismas preguntas. Gini es un poco más barato

de calcular (no tiene logaritmos) y es la opción por defecto en scikit-learn. Usaremos Gini en el resto de las notas.

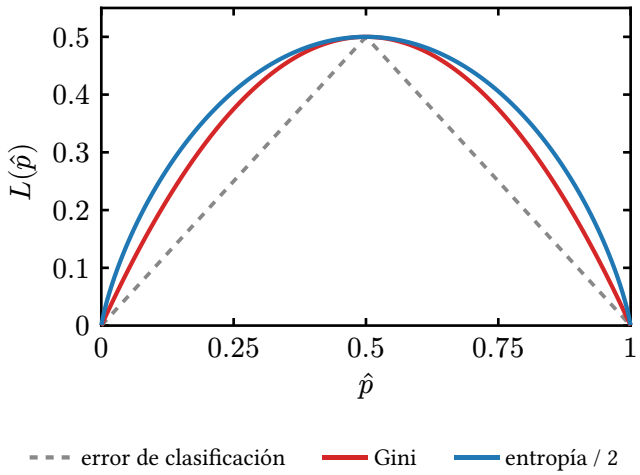


Figura 6: Las tres medidas de impureza para clasificación binaria, en función de la proporción de positivos. La entropía está dividida entre 2 para que las tres tengan el mismo máximo. Gini y la entropía son curvas suaves y estrictamente cóncavas. El error de clasificación es una pirámide.

2.6. Cuántas preguntas hay que probar

En cada nodo buscamos

$$(j, t) = \operatorname*{argmax}_{j,t} \text{ganancia}(j, t).$$

Parece que hay infinitos umbrales t posibles, pero no es así. Si ordenamos los ejemplos de la región según la feature j, cualquier umbral entre dos valores consecutivos divide los ejemplos de la misma forma. Con m ejemplos hay solo $m - 1$ divisiones distintas por feature, y se suele tomar como umbral el punto medio entre los dos valores consecutivos. En total hay que evaluar a lo sumo $d(m - 1)$ preguntas, y el algoritmo simplemente las prueba todas.

Encontrar el árbol óptimo (el de menor error con un número dado de hojas) es un problema computacionalmente intratable (NP-completo). Por eso usamos el algoritmo codicioso: no garantiza el mejor árbol, pero es rápido y funciona bien.

3. Parte 3: Regresión, variables categóricas y regularización

3.1. Árboles de regresión

Los árboles también sirven para regresión. Cambiemos el ejemplo: en lugar de predecir si se puede esquiar, queremos predecir cuántos centímetros de nieve caen por día, con las mismas dos features, el mes y la latitud. La Figura 7 muestra el mismo plano de la Figura 1, pero en lugar de $+$ y $-$ cada ejemplo tiene un número: los centímetros de nieve de ese lugar y ese mes. En los inviernos de cada hemisferio los números son grandes, y en el resto son ceros.

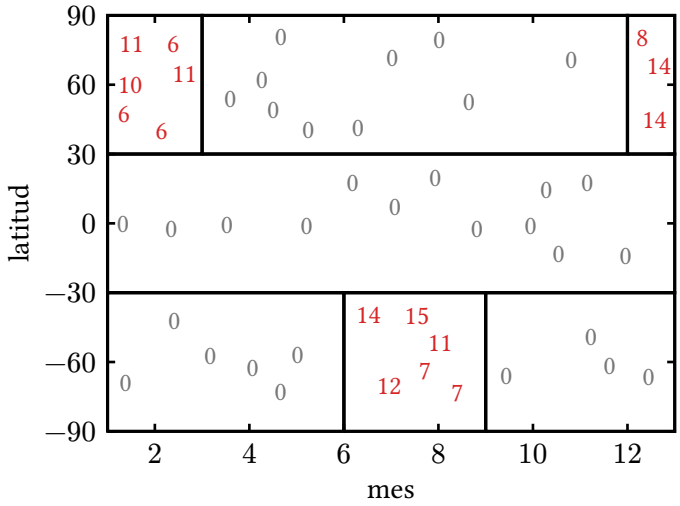


Figura 7: El problema del esquí como regresión, con 48 ejemplos: cada número es la cantidad de nieve (en cm por día) de un ejemplo. Las líneas son las regiones del árbol de la Figura 3. Cada hoja predice el promedio de los números de su región: 8.3, 12 y 11 cm en las tres regiones de invierno (arriba a la izquierda, arriba a la derecha y abajo al centro), y 0 en las demás.

Solo cambian dos cosas respecto a la clasificación. Primero, cada hoja predice el promedio de las etiquetas de los ejemplos que caen en ella:

$$w_j = \frac{1}{|R_j|} \sum_{i:\mathbf{x}^{(i)} \in R_j} y^{(i)}.$$

Segundo, la impureza de una región es el error cuadrático medio de esa predicción, que es la varianza de las etiquetas en la región:

$$L_{\text{cuadrática}}(R_j) = \frac{1}{|R_j|} \sum_{i:\mathbf{x}^{(i)} \in R_j} \left(y^{(i)} - w_j\right)^2.$$

La ganancia se define igual que antes, con los mismos pesos.

Para ver mejor la forma de la predicción, quedémonos con una sola feature: la nieve en un lugar del hemisferio norte, según el mes. En la Figura 8 se ve el resultado: una función constante a trozos, con un escalón por cada hoja. Con más hojas, la escalera sigue más de cerca a los datos.

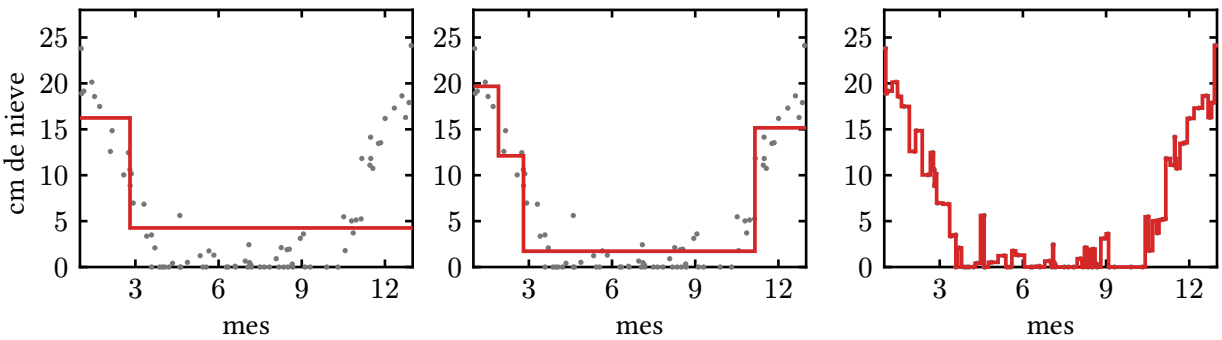


Figura 8: Árboles de regresión sobre 80 ejemplos, con profundidad 1 (2 hojas), 2 (4 hojas) y sin límite (68 hojas). El error cuadrático medio de entrenamiento es 32.7, 6.9 y 0. El árbol sin límite pasa exactamente por cada ejemplo, incluido el ruido.

Hay una relación directa entre Gini y la pérdida cuadrática. En clasificación binaria, codifiquemos los positivos como $y = 1$ y los negativos como $y = 0$. El promedio de las etiquetas en la región es $\hat{p}$, y su varianza es

$$\frac{1}{|R|}\sum_i \left(y^{(i)} - \hat{p}\right)^2 = \hat{p}(1-\hat{p})^2 + (1-\hat{p})\hat{p}^2 = \hat{p}(1-\hat{p}),$$

porque una fracción $\hat{p}$ de los ejemplos tiene error $1-\hat{p}$ y el resto tiene error $-\hat{p}$. Es exactamente la mitad de la impureza de Gini. Es decir, un árbol de clasificación con Gini es un árbol de regresión que predice la probabilidad de la clase positiva con pérdida cuadrática. Esta idea, que todo se reduce a ajustar promedios con pérdida cuadrática, reaparecerá en boosting.

3.2. Variables categóricas

Los árboles manejan variables categóricas de forma natural. En lugar de comparar con un umbral, la pregunta es si la categoría pertenece a un subconjunto. Por ejemplo, si en lugar de la latitud tenemos la ubicación como una de tres categorías, hemisferio norte (N), Ecuador (E) y hemisferio sur (S), una pregunta posible es "¿ubicación $\in \{N\}$?", y otra es "¿ubicación $\in \{N, S\}$?".

El problema es que, con q categorías, hay $2^{q-1}-1$ formas de dividirlas en dos grupos. Con $q = 30$, son más de 500 millones. Para clasificación binaria hay un truco: se ordenan las categorías según su proporción de positivos y se prueban solo los $q-1$ cortes de esa lista ordenada. Se puede demostrar que el mejor corte (con Gini o entropía) está entre ellos. En otros casos, con muchas categorías, se suelen agrupar o codificar de otra forma.

3.3. Regularización y poda

Si dejamos que el árbol crezca sin restricciones, sigue dividiendo hasta que cada hoja es pura, lo que en el peor caso significa una hoja por ejemplo. El error de entrenamiento es cero, pero el árbol memorizó los datos, incluido el ruido (Figura 8, derecha). Los árboles de decisión son modelos de alta varianza: pequeños cambios en los datos producen árboles muy distintos.

Para regularizarlos se usan reglas de parada:

- **Tamaño mínimo de hoja:** no dividir una región si quedaría con menos de cierto número de ejemplos.
- **Profundidad máxima:** limitar el número de preguntas desde la raíz hasta una hoja.
- **Número máximo de hojas.**
- **Ganancia mínima:** no dividir si la mejor ganancia es menor que un umbral.

La última parece la más natural, pero suele ser una mala idea. Hay preguntas que por sí solas no ayudan, pero que hacen posible una segunda pregunta muy buena. En el problema del esquí, preguntar solo por el mes no separa casi nada (en el mismo mes se puede esquiar en un hemisferio pero no en el otro), pero combinado con la latitud es muy útil. La Figura 9 muestra el caso extremo.

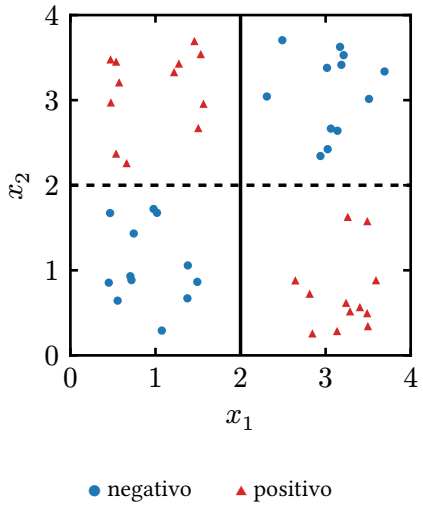


Figura 9: Datos en los que cada clase ocupa dos esquinas opuestas. Cualquier pregunta individual deja los dos hijos casi tan mezclados como el padre: la mejor ganancia de Gini posible en la raíz es 0.056 (el padre tiene impureza 0.5). Pero después de la pregunta $x_1 < 2$, la pregunta $x_2 < 2$ en cada hijo separa todo perfectamente.

La alternativa recomendada es la poda (pruning). Primero se deja crecer el árbol completo. Luego se recorren las hojas de abajo hacia arriba y se eliminan las divisiones que no mejoran el error sobre un conjunto de validación, juntando las dos hojas en una sola. Así el árbol puede descubrir interacciones como la de la Figura 9, y solo después decidimos qué partes valen la pena.

3.4. Evaluar un clasificador: exactitud y matriz de confusión

Hasta ahora hemos medido qué tan bueno es un clasificador con su exactitud (accuracy), la fracción de ejemplos que clasifica bien, o su complemento, el error. La exactitud es fácil de interpretar, pero puede engañar.

Volvamos al esquí, ahora con 400 ejemplos, de los que separamos 120 para prueba. Solo 23 de esos 120 ejemplos son positivos: en la mayoría de los lugares y meses no se puede esquiar. Un "modelo" que ignora los datos y siempre responde "no se puede esquiar" acierta en todos los negativos, así que su exactitud es 80.8%. Parece buena, pero el modelo no sirve para nada: nunca encuentra un día para esquiar. Cuando una clase es mucho más frecuente que la otra (se dice que las clases están desbalanceadas), la exactitud por sí sola no basta.

La matriz de confusión separa los aciertos y los errores según la clase. Para clasificación binaria es una tabla de 2×2: las filas son la clase real y las columnas la clase predicha.

- **Verdaderos positivos** (VP): positivos que el modelo predice como positivos. Días buenos para esquiar que el modelo detecta.
- **Verdaderos negativos** (VN): negativos predichos como negativos.
- **Falsos positivos** (FP): negativos que el modelo predice como positivos. Vamos a esquiar y no hay nieve.
- **Falsos negativos** (FN): positivos que el modelo predice como negativos. Nos perdemos un buen día para esquiar.

La exactitud es la suma de la diagonal dividida entre el total: $(\mathrm{VP} + \mathrm{VN})/(\mathrm{VP} + \mathrm{VN} + \mathrm{FP} + \mathrm{FN})$. La Figura 10 compara las matrices del modelo que siempre dice "no" y de un árbol de profundidad 3.

Siempre “no”

	predice “no”	predice “sí”
real “no”	**97** VN	**0** FP
real “sí”	**23** FN	**0** VP

Árbol de profundidad 3

	predice “no”	predice “sí”
real “no”	**97** VN	**0** FP
real “sí”	**9** FN	**14** VP

Figura 10: Matrices de confusión sobre los 120 ejemplos de prueba. Las filas son la clase real y las columnas la predicha. A la izquierda, el modelo que siempre predice “no” tiene exactitud 80.8%, pero cero verdaderos positivos. A la derecha, un árbol de profundidad 3 tiene exactitud 92.5%: no comete ningún falso positivo, pero se le escapan 9 de los 23 días buenos.

De la matriz se leen dos medidas que responden preguntas más específicas que la exactitud:

$$\text{precisión} = \frac{\text{VP}}{\text{VP} + \text{FP}}, \qquad \text{sensibilidad} = \frac{\text{VP}}{\text{VP} + \text{FN}}.$$

La precisión (precision) responde: de los días en que el modelo dijo “sí”, ¿qué fracción eran buenos? La sensibilidad (recall) responde: de los días buenos, ¿qué fracción detectó el modelo? Para el árbol de la figura, la precisión es 100% y la sensibilidad es 60.9%: cada vez que dice “sí” acierta, pero detecta poco más de la mitad de los días buenos. El modelo que siempre dice “no” tiene sensibilidad 0 (y su precisión ni siquiera está definida, porque nunca dice “sí”).

Qué medida importa más depende del problema. En un examen médico para detectar una enfermedad, un falso negativo (no detectar a un enfermo) suele ser mucho más grave que un falso positivo, y se busca una sensibilidad alta. En un filtro de spam, un falso positivo (mandar a spam un correo importante) es peor, y se busca una precisión alta.

3.5. Ventajas y desventajas

Ventajas:

- **Fáciles de explicar e interpretar.** Se puede dibujar el árbol y ver exactamente qué preguntas llevan a cada predicción.
- **Manejan variables categóricas y mezclas de variables de distinto tipo** sin preprocesamiento.
- **No les afectan las transformaciones monótonas de las features.** Como solo comparan con umbrales, medir la temperatura en grados Celsius o Fahrenheit, o tomar logaritmo de una feature, produce exactamente el mismo árbol. En particular, no hace falta estandarizar, a diferencia de las SVM con kernel gaussiano.
- **Rápidos** de entrenar y de evaluar.

Desventajas:

- **Alta varianza.** Tienden al overfitting, y pequeños cambios en los datos producen árboles muy distintos.
- **Les cuesta capturar estructuras aditivas.** Una frontera tan simple como $x_2 = x_1$, que la regresión logística encuentra de inmediato, obliga al árbol a aproximarla con una escalera de muchas preguntas (Figura 11).
- Por estas dos razones, un árbol solo suele tener **baja precisión predictiva**.

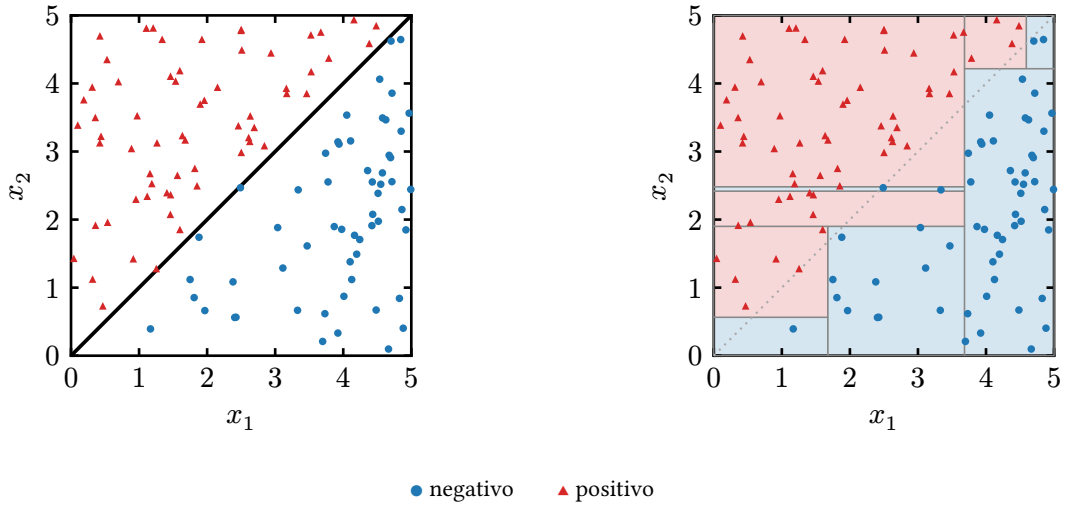


Figura 11: Los positivos están arriba de la recta $x_2 = x_1$. A la izquierda, un clasificador lineal la encuentra con un solo parámetro de pendiente. A la derecha, un árbol completo necesita 9 hojas para separar los ejemplos de entrenamiento, y su frontera es una escalera que solo aproxima la diagonal.

Lo interesante es que la primera desventaja tiene solución. El resto de la clase trata de cómo combinar muchos árboles para reducir su varianza (bagging y random forests) o su sesgo (boosting).

En scikit-learn, las clases `DecisionTreeClassifier` y `DecisionTreeRegressor` implementan estos árboles. El criterio por defecto en clasificación es `criterion="gini"`. Los parámetros `max_depth`, `min_samples_leaf` y `max_leaf_nodes` corresponden a las reglas de parada, y `ccp_alpha` a una forma de poda.

4. Parte 4: Ensamblaje: bagging y random forests

4.1. Por qué promediar modelos

Un ensamblaje (ensemble) es un modelo que combina las predicciones de varios modelos. La razón por la que funciona viene de la estadística básica.

Sean $X_1, \ldots, X_B$ variables aleatorias independientes e idénticamente distribuidas (i.i.d.), cada una con varianza σ^2. La varianza de su promedio $\overline{X} = \frac{1}{B}\sum_b X_b$ es

$$\operatorname{Var}\left(\overline{X}\right) = \frac{\sigma^2}{B}.$$

Promediar B variables independientes divide la varianza entre B. Si pensamos en cada X_b como la predicción de un modelo entrenado con un conjunto de datos distinto, promediar muchos modelos reduce la varianza tanto como queramos.

El problema es que los modelos no son independientes: se entrenan con los mismos datos, o con datos parecidos. Supongamos que las X_b siguen teniendo varianza σ^2, pero que cada par tiene correlación ρ, es decir, $\operatorname{Cov}(X_b, X_{b'}) = \rho\sigma^2$ para $b \neq b'$. Entonces

$$\operatorname{Var}\left(\overline{X}\right) = \frac{1}{B^2}\sum_{b=1}^{B}\sum_{b'=1}^{B}\operatorname{Cov}(X_b, X_{b'}) = \frac{1}{B^2}\left(B\sigma^2 + B(B-1)\rho\sigma^2\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

(En la doble suma hay B términos con $b = b'$, que son varianzas, y $B(B-1)$ con $b \neq b'$, que son covarianzas.)

Al aumentar B, el segundo término desaparece, pero el primero no: la varianza nunca baja de $\rho\sigma^2$ (Figura 12). Esto dice exactamente qué hay que hacer para que un ensamblaje funcione:

1. Usar muchos modelos (B grande), para eliminar el segundo término.
2. Hacer que los modelos estén poco correlacionados (ρ pequeño), para reducir el primero.

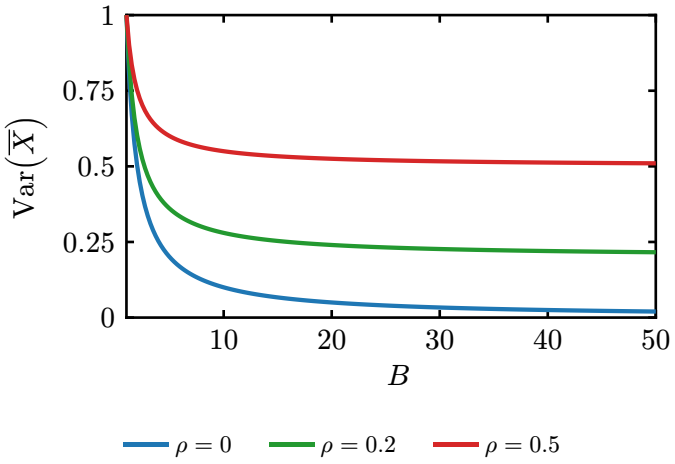


Figura 12: Varianza del promedio de B modelos con varianza $\sigma^2 = 1$, para tres valores de la correlación ρ. Con modelos independientes la varianza tiende a cero. Con modelos correlacionados se estanca en $\rho\sigma^2$.

Una forma de obtener modelos poco correlacionados es usar algoritmos distintos: por ejemplo, promediar una SVM, una regresión logística y un árbol. Es muy común en competencias como las de Kaggle. Pero también hay métodos que generan automáticamente muchos modelos distintos a partir de un solo algoritmo. El primero es bagging.

4.2. Bootstrap

Idealmente, para obtener modelos independientes, tomaríamos varios conjuntos de entrenamiento independientes $S_1, S_2, \ldots, S_B$ de la población, y entrenaríamos un modelo con cada uno. Pero solo tenemos un conjunto de entrenamiento S, con m ejemplos.

El bootstrap es una técnica de la estadística para simular esa situación. Supone que el conjunto de entrenamiento S representa bien a la población, y genera conjuntos nuevos $Z_1, \ldots, Z_B$ sacando m ejemplos de S al azar con reemplazo. Como el muestreo es con reemplazo, cada Z_b contiene algunos ejemplos repetidos y le faltan otros (Tabla 2).

Muestra	Ejemplos	Distintos
Z_1	1, 1, 3, 5, 6, 7, 7, 9, 9, 10	7
Z_2	1, 1, 1, 2, 2, 3, 4, 5, 6, 10	7
Z_3	2, 2, 3, 3, 3, 5, 7, 8, 9, 10	7

Tabla 2: Tres muestras bootstrap de un conjunto de 10 ejemplos, numerados del 1 al 10. Cada muestra tiene 10 elementos, pero solo 7, 7 y 7 ejemplos distintos.

¿Qué fracción de los ejemplos originales aparece en cada muestra? En cada uno de los m sorteos, un ejemplo dado no sale con probabilidad $1 - 1/m$. La probabilidad de que no salga en ninguno es $(1 - 1/m)^m$, así que la probabilidad de que aparezca al menos una vez es

$$1 - \left(1 - \frac{1}{m}\right)^m \to 1 - e^{-1} \approx 0.632 \quad \text{cuando } m \to \infty.$$

Es decir, cada muestra bootstrap contiene aproximadamente dos tercios de los ejemplos distintos del conjunto original. (Para $m = 10$, la fracción esperada es 0.651.)

4.3. Bagging

Bagging (de bootstrap aggregating) consiste en:

1. Generar B muestras bootstrap $Z_1, ..., Z_B$.
2. Entrenar un modelo G_b con cada muestra Z_b.
3. Promediar las predicciones:

$$G(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} G_b(\mathbf{x}).$$

En clasificación, se promedian las probabilidades que predice cada modelo, o cada modelo vota por una clase y gana la mayoría.

¿Qué efecto tiene sobre el sesgo y la varianza?

- **Reduce la varianza.** Los modelos se entrenan con datos distintos, así que están menos correlacionados que si usaran todos el mismo conjunto: el bootstrap reduce ρ. Y al aumentar B, el término $(1 - \rho)\sigma^2/B$ desaparece. Un detalle importante: **aumentar B no produce overfitting.** Más modelos solo acercan la varianza a su límite $\rho\sigma^2$. El único costo de un B grande es computacional.
- **Aumenta un poco el sesgo.** Cada modelo ve solo alrededor del 63% de los ejemplos distintos, así que individualmente es algo peor que un modelo entrenado con todos los datos.

Bagging funciona especialmente bien con árboles de decisión. Un árbol completo tiene sesgo bajo y varianza alta: exactamente el tipo de modelo que se beneficia de reducir la varianza a cambio de un poco de sesgo. Además, los árboles son muy sensibles a los datos, así que las muestras bootstrap producen árboles bastante distintos entre sí, con ρ relativamente bajo. Con un modelo muy estable, como una regresión lineal, todas las muestras producirían casi el mismo modelo y bagging no ganaría casi nada.

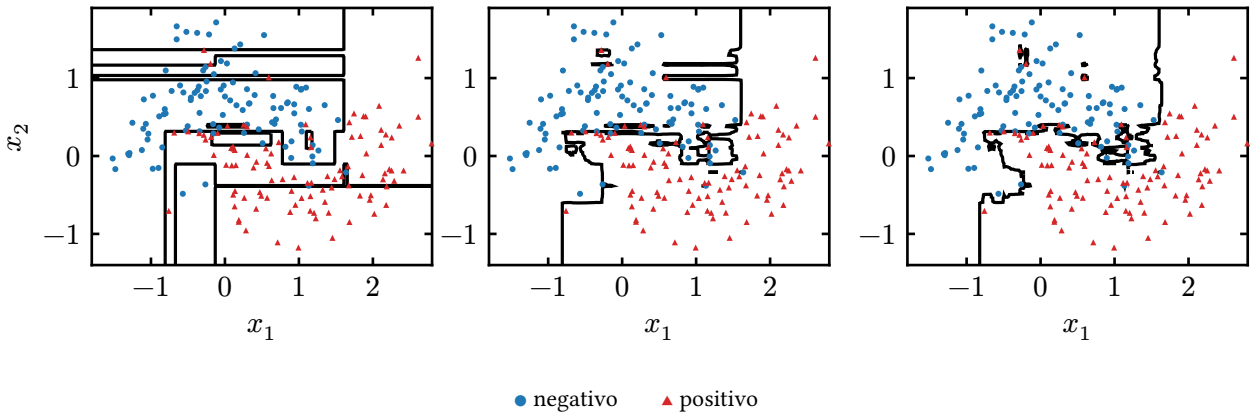


Figura 13: Fronteras de decisión con 200 ejemplos ruidosos: un solo árbol completo (izquierda), bagging con $B = 300$ árboles (centro) y un random forest con $B = 300$ árboles que en cada división consideran una sola feature al azar (derecha). Los tres clasifican correctamente todos los ejemplos de entrenamiento, pero la exactitud sobre datos de prueba es 79.4%, 84.9% y 85.9%. El árbol individual tiene una frontera llena de entrantes que siguen el ruido. Al promediar, la frontera se suaviza.

4.4. Random forests

Bagging reduce ρ, pero no tanto como quisiéramos. Supongamos que hay una feature mucho más predictiva que las demás. Entonces casi todos los árboles, aunque se entrenen con muestras

bootstrap distintas, van a escoger esa misma feature en la raíz, y probablemente las mismas en los niveles siguientes. Los árboles terminan pareciéndose mucho, y ρ queda alto.

Un random forest (bosque aleatorio) es bagging con árboles más una fuente adicional de aleatoriedad: **en cada división, el árbol solo puede escoger entre un subconjunto de $k < d$ features sorteadas al azar.** El sorteo se repite en cada nodo de cada árbol. En el ejemplo del esquí, un árbol podría estar obligado a preguntar primero por el mes, porque en ese nodo la latitud no estaba disponible, y otro árbol a preguntar primero por la latitud.

Así, la feature dominante no puede aparecer en todos los árboles en la misma posición, y los árboles quedan más distintos entre sí. Cada árbol individual es un poco peor (sesgo un poco mayor), pero la correlación ρ baja, y el promedio suele ser mejor que el de bagging. Valores típicos son $k \approx \sqrt{d}$ en clasificación y $k \approx d/3$ en regresión.

En scikit-learn, `RandomForestClassifier` y `RandomForestRegressor` implementan random forests. El parámetro `n_estimators` es B y `max_features` es k. Con `max_features=None` se considera todas las features en cada división, y se obtiene bagging. Random forests es probablemente el mejor algoritmo "listo para usar": funciona bien con los parámetros por defecto y es difícil que haga mucho overfitting.

5. Parte 5: Boosting

5.1. Idea: combinar modelos débiles

Bagging y random forests promedian modelos que se entrenan en paralelo, cada uno sin saber nada de los demás, y reducen la varianza. Boosting hace lo contrario: entrena los modelos uno después de otro, y cada modelo nuevo trata de corregir los errores de los anteriores. El objetivo principal es reducir el sesgo.

Por eso, en lugar de árboles completos (sesgo bajo y varianza alta), boosting usa modelos con sesgo alto y varianza baja. El caso extremo es el tocón (decision stump), un árbol con una sola pregunta y dos hojas. Un tocón por sí solo es un modelo muy débil, apenas mejor que adivinar. La pregunta de boosting es: ¿se puede construir un modelo fuerte sumando muchos modelos débiles?

5.2. AdaBoost

AdaBoost (adaptive boosting) fue el primer algoritmo de boosting exitoso. Usamos etiquetas $y \in \{-1, 1\}$, como en las SVM. La idea es mantener un peso w_i para cada ejemplo de entrenamiento y entrenar cada modelo nuevo dándole más importancia a los ejemplos en los que los anteriores se equivocaron.

1. Inicializar los pesos: $w_i = 1/m$ para $i = 1, \ldots, m$.
2. Para $b = 1, \ldots, B$:
   1. Entrenar un clasificador G_b con los ejemplos ponderados por los pesos w_i. (Para un árbol, basta con usar las proporciones ponderadas al calcular la impureza: cada ejemplo cuenta w_i veces.)
   2. Calcular su error ponderado:
   $$\text{err}_b = \frac{\sum_{i=1}^{m} w_i \mathbb{1}\{y^{(i)} \neq G_b(\mathbf{x}^{(i)})\}}{\sum_{i=1}^{m} w_i}.$$
   3. Calcular su peso en la votación: $\alpha_b = \log\left(\frac{1-\text{err}_b}{\text{err}_b}\right)$.
   4. Aumentar el peso de los ejemplos mal clasificados: $w_i \leftarrow w_i \exp\big(\alpha_b \mathbb{1}\{y^{(i)} \neq G_b(\mathbf{x}^{(i)})\}\big)$.
3. El clasificador final es una votación ponderada:

$$G(\mathbf{x}) = \text{signo}\left(\sum_{b=1}^{B} \alpha_b G_b(\mathbf{x})\right).$$

Algunas observaciones:

- Si $\text{err}_b < 1/2$, es decir, si el clasificador es mejor que adivinar, entonces $\alpha_b > 0$. Cuanto menor es el error, mayor es α_b, y más pesa ese clasificador en la votación final.
- Los ejemplos bien clasificados conservan su peso, y los mal clasificados lo multiplican por $e^{\alpha_b} = (1 - \text{err}_b)/\text{err}_b$. Con un error del 10%, por ejemplo, el peso de cada error se multiplica por 9. Así, el siguiente clasificador se concentra en los ejemplos difíciles.

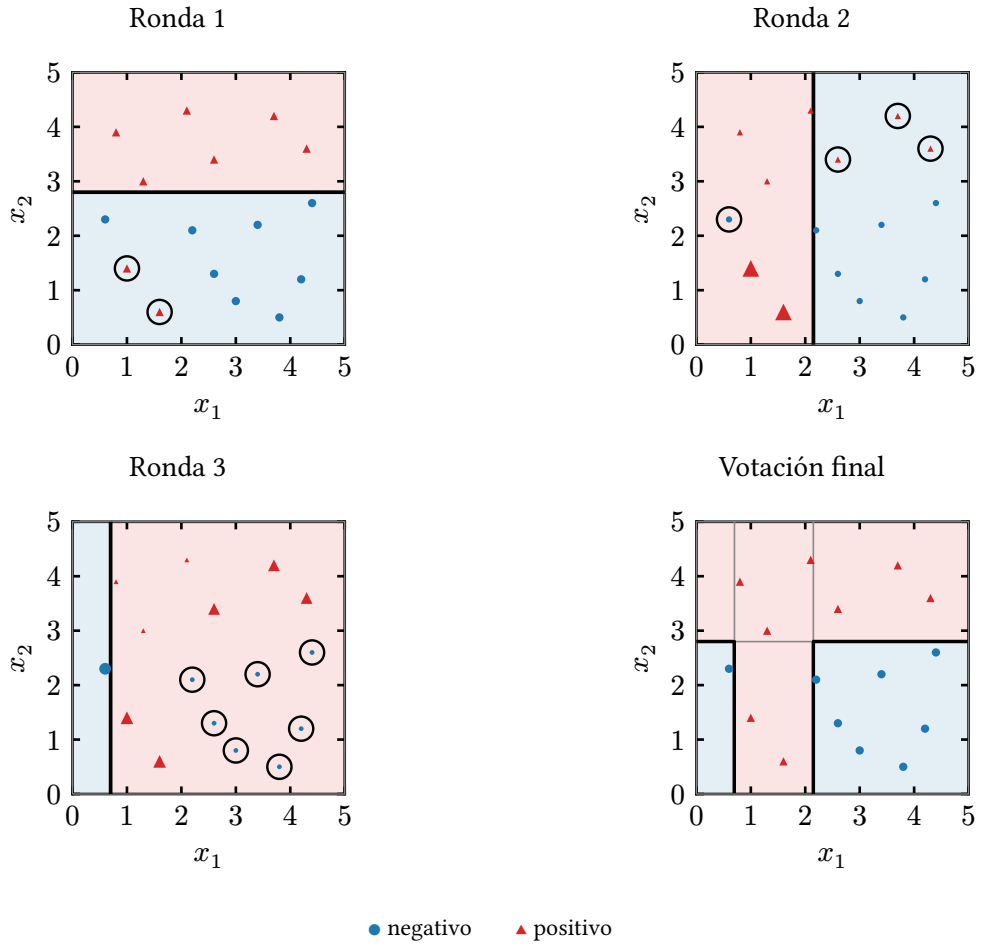


Figura 14: Tres rondas de AdaBoost con tocones. El tamaño de cada punto es proporcional a la raíz de su peso. En la ronda 1 todos pesan lo mismo, y el tocón (una pregunta sobre x_2) se equivoca en los dos ejemplos marcados con un círculo, con $\text{err}_1 = 0.125$ y $\alpha_1 = 1.95$. En la ronda 2 esos ejemplos pesan 7 veces más que los demás, y el nuevo tocón (una pregunta sobre x_1) los clasifica bien a cambio de equivocarse en otros, de menor peso. Abajo a la derecha, la votación ponderada de los tres tocones, que ya clasifica correctamente todos los ejemplos.

La frontera final de la Figura 14 no es un rectángulo ni una recta: sumando tres preguntas muy simples obtuvimos una frontera que ninguna de ellas podía producir sola. Con más rondas se pueden obtener fronteras muy complejas. Por eso boosting reduce el sesgo.

La Figura 15 compara los métodos de esta clase en un problema con 10 features. Un tocón solo tiene un error de prueba de 45.9%, casi como adivinar. Un árbol completo tiene 24.5%. Bagging y random forests mejoran el árbol completo y se estabilizan a medida que crece B. AdaBoost con tocones empieza muy mal, pero sigue mejorando ronda tras ronda, y termina siendo el mejor de los tres.

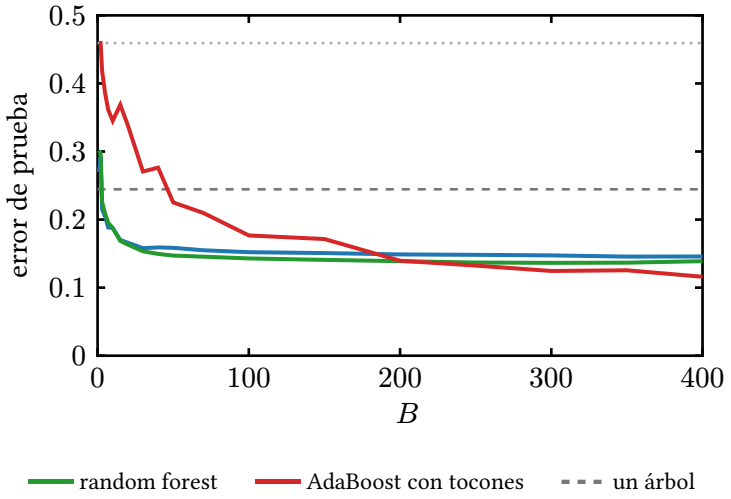


Figura 15: Error de prueba en función del número de modelos B, con 2000 ejemplos de entrenamiento y 10000 de prueba. Los datos son un problema clásico del libro *The Elements of Statistical Learning* (Hastie, Tibshirani y Friedman): 10 features gaussianas, y la clase depende de si la suma de sus cuadrados supera un umbral. Las rectas horizontales son un solo tocón y un solo árbol completo.

AdaBoost se puede derivar como un caso particular de una idea más general, que es la base de los métodos de boosting que se usan hoy: gradient boosting y XGBoost. Para llegar a ellos, conviene empezar por regresión.

5.3. Boosting con pérdida cuadrática: ajustar los residuos

Volvamos a los datos de nieve de la Parte 3. Queremos construir un modelo como una suma de árboles pequeños:

$$F_B(\mathbf{x}) = F_0(\mathbf{x}) + h_1(\mathbf{x}) + h_2(\mathbf{x}) + \cdots + h_B(\mathbf{x}),$$

donde F_0 es un modelo inicial muy simple (por ejemplo, el promedio de todas las etiquetas) y cada h_b es un árbol de regresión. Igual que en AdaBoost, construimos el modelo un árbol a la vez: $F_b = F_{b-1} + h_b$.

¿Qué árbol h_b hay que agregar? Con pérdida cuadrática, queremos que $F_{b-1}\big(\mathbf{x}^{(i)}\big) + h_b\big(\mathbf{x}^{(i)}\big)$ se acerque lo más posible a $y^{(i)}$. Es decir, queremos que

$$h_b\big(\mathbf{x}^{(i)}\big) \approx y^{(i)} - F_{b-1}\big(\mathbf{x}^{(i)}\big) = r_i.$$

La diferencia r_i entre la etiqueta y la predicción actual se llama residuo: es lo que al modelo actual todavía le falta explicar. Así que cada árbol nuevo es simplemente un árbol de regresión entrenado para predecir los residuos del modelo actual (Figura 16). Es la misma idea de AdaBoost, "concentrarse en los errores del modelo anterior", pero en lugar de repesar los ejemplos cambiamos las etiquetas.

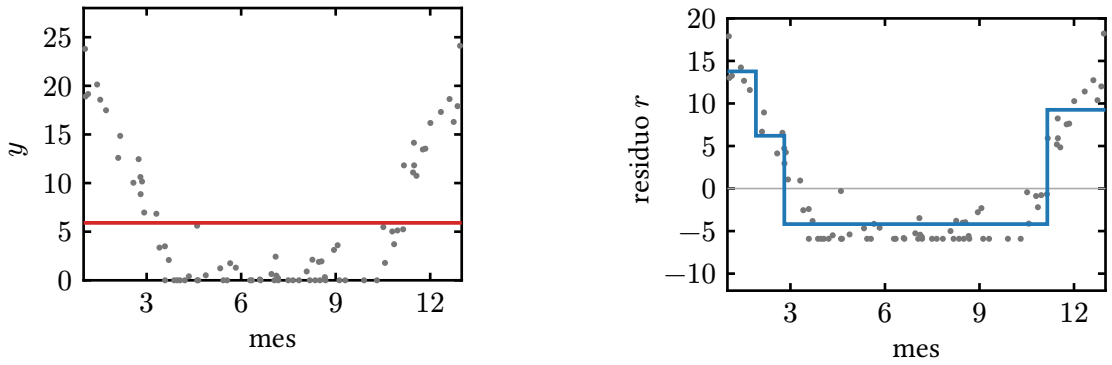


Figura 16: El primer paso de boosting con los datos de nieve. A la izquierda, el modelo inicial F_0 es el promedio de las etiquetas (5.9 cm). A la derecha, los residuos $r_i = y^{(i)} - F_0(\mathbf{x}^{(i)})$ y el primer árbol h_1 (de profundidad 2), entrenado para predecirlos.

En la práctica no se suma el árbol completo, sino una fracción:

$$F_b(\mathbf{x}) = F_{b-1}(\mathbf{x}) + \nu h_b(\mathbf{x}),$$

donde $0 < \nu \leq 1$ se llama tasa de aprendizaje (learning rate o shrinkage), y valores típicos son $\nu = 0.1$ o 0.3. Con ν pequeño cada árbol corrige solo una parte del error y deja el resto para los siguientes, lo que hace el proceso más lento pero menos propenso al overfitting. La Figura 17 muestra cómo se va formando el modelo.

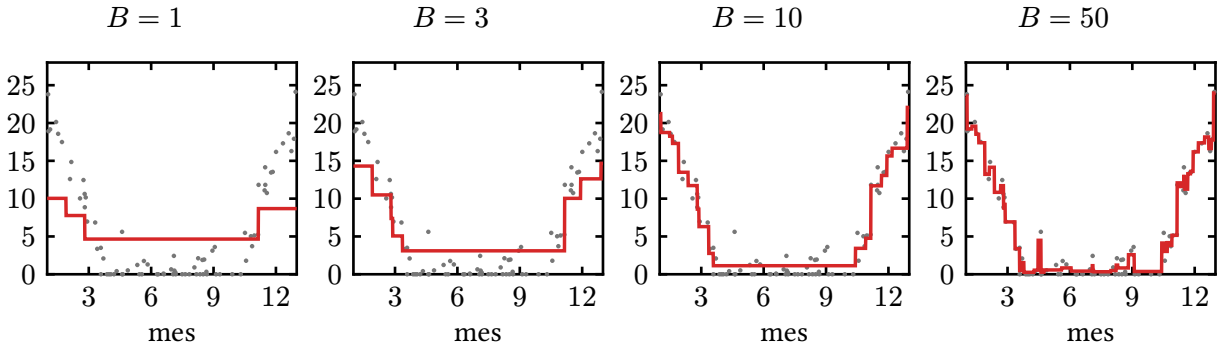


Figura 17: El modelo F_B después de $B = 1$, 3, 10 y 50 árboles de profundidad 2, con $\nu = 0.3$. Cada árbol tiene solo 4 hojas, pero la suma de muchos de ellos aproxima la forma de los datos.

A diferencia de bagging, en boosting **aumentar B sí puede producir overfitting**: cada árbol nuevo ajusta los residuos que quedan, y cuando ya solo queda ruido, el modelo empieza a memorizarlo. En la Figura 18 el error de entrenamiento baja hasta casi cero, mientras que el error de prueba llega a un mínimo y luego empieza a subir. Por eso B se escoge con validación, o se detiene el entrenamiento cuando el error de validación deja de mejorar (early stopping).

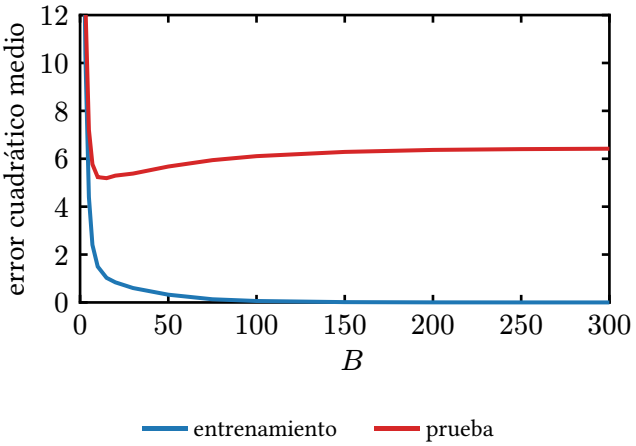


Figura 18: Error cuadrático medio de entrenamiento y de prueba de boosting con los datos de nieve, en función del número de árboles (los primeros valores, cercanos a 28, quedan fuera de la gráfica). El error de prueba es mínimo con $B = 15$ (5.2) y luego sube hasta 6.4 con $B = 300$, mientras que el de entrenamiento sigue bajando.

5.4. Gradient boosting

Ajustar residuos tiene sentido con pérdida cuadrática, pero ¿qué hacemos con otras pérdidas, como la de la regresión logística? La clave es ver los residuos de otra forma.

Pensemos en las predicciones del modelo sobre los ejemplos de entrenamiento como m números, $\hat{y}_i = F_{b-1}(\mathbf{x}^{(i)})$, y en la pérdida total como función de ellos:

$$\mathcal{L} = \sum_{i=1}^{m} \ell(y^{(i)}, \hat{y}_i).$$

Si pudiéramos mover cada $\hat{y}_i$ libremente, para reducir la pérdida daríamos un paso de descenso por gradiente: moveríamos cada $\hat{y}_i$ en la dirección de

$$r_i = -\frac{\partial \ell(y^{(i)}, \hat{y}_i)}{\partial \hat{y}_i}.$$

Con la pérdida cuadrática $\ell(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$, la derivada es $-(y - \hat{y})$, y $r_i = y^{(i)} - \hat{y}_i$ es exactamente el residuo. Es decir, ajustar los residuos es dar un paso de descenso por gradiente.

Pero no podemos mover los $\hat{y}_i$ libremente: necesitamos una función que también sirva para puntos nuevos. La solución es entrenar un árbol de regresión h_b para predecir los r_i, y sumarlo con la tasa de aprendizaje, $F_b = F_{b-1} + \nu h_b$. El árbol "generaliza" el paso de gradiente a todo el espacio.

Esto es gradient boosting: el mismo algoritmo de la sección anterior, pero en lugar de los residuos, cada árbol se entrena con los gradientes negativos de la pérdida, que se llaman pseudo-residuos. Para cambiar de problema basta con cambiar la pérdida. Por ejemplo, en clasificación binaria con $y \in \{0, 1\}$, si el modelo predice el logit $\hat{y}$ y la probabilidad es $\sigma(\hat{y})$, con la pérdida de la regresión logística el pseudo-residuo resulta ser $r_i = y^{(i)} - \sigma(\hat{y}_i)$: la diferencia entre la etiqueta y la probabilidad predicha. Otra vez, "lo que falta explicar".

En scikit-learn, `GradientBoostingRegressor` y `GradientBoostingClassifier` implementan gradient boosting con árboles. Los parámetros principales son `n_estimators` (B), `learning_rate` (ν) y `max_depth` (la profundidad de cada árbol, típicamente entre 2 y 6).

5.5. XGBoost

XGBoost (extreme gradient boosting) es una implementación de gradient boosting con árboles que, desde que apareció en 2014, ha ganado una enorme cantidad de competencias y se usa constantemente en la industria. Es un caso raro en este curso: un algoritmo que está muy cerca del estado del arte para datos tabulares y que podemos entender con lo que ya sabemos. Sus diferencias principales con gradient boosting son tres.

1. Regularización. XGBoost minimiza la pérdida más una penalización por la complejidad de cada árbol:

$$\sum_{i=1}^{m} \ell\big(y^{(i)}, \hat{y}_i\big) + \sum_{b=1}^{B} \left(\gamma J_b + \frac{1}{2}\lambda \sum_{j=1}^{J_b} w_{bj}^2 \right),$$

donde J_b es el número de hojas del árbol b y w_{bj} son los valores de sus hojas. El término γJ penaliza los árboles con muchas hojas, y $\frac{1}{2}\lambda \sum w_j^2$ es una penalización L_2, como en la regresión ridge, sobre los valores de las hojas.

Veamos qué efecto tiene con pérdida cuadrática. Al agregar el árbol b, en cada hoja R_j tenemos que escoger un valor w_j que se suma a la predicción de todos los ejemplos de la hoja. Si r_i son los residuos, hay que minimizar

$$\sum_{i \in R_j} \frac{1}{2}\big(r_i - w_j\big)^2 + \frac{1}{2}\lambda w_j^2.$$

Derivando respecto a w_j e igualando a cero, $-\sum_{i \in R_j}\big(r_i - w_j\big) + \lambda w_j = 0$, de donde

$$w_j = \frac{\sum_{i \in R_j} r_i}{n_j + \lambda},$$

donde n_j es el número de ejemplos en la hoja. Sin regularización ($\lambda = 0$), w_j es el promedio de los residuos de la hoja, como en un árbol de regresión normal. Con $\lambda > 0$, es un promedio "encogido" hacia cero. El efecto es mayor en las hojas con pocos ejemplos: si una hoja tiene un solo ejemplo con residuo 10, con $\lambda = 1$ el valor de la hoja es $10/2 = 5$; si tiene 100 ejemplos con residuo promedio 10, el valor es $1000/101 \approx 9.9$. Es decir, XGBoost desconfía de las hojas que se basan en pocos ejemplos.

Llamemos $S_j = \sum_{i \in R_j} r_i$ a la suma de los residuos de la hoja, así que $w_j = S_j/\big(n_j + \lambda\big)$. Sustituyendo este w_j en la expresión anterior, el valor mínimo es

$$\frac{1}{2}\sum_{i \in R_j} r_i^2 - \frac{1}{2}\frac{S_j^2}{n_j + \lambda}.$$

El primer término es el mismo sin importar cómo dividamos los ejemplos de la hoja, así que la ganancia de dividir una hoja en dos hijos L y R es

$$\text{ganancia} = \frac{1}{2}\left(\frac{S_L^2}{n_L + \lambda} + \frac{S_R^2}{n_R + \lambda} - \frac{\big(S_L + S_R\big)^2}{n_L + n_R + \lambda} \right) - \gamma.$$

Es el mismo esquema de la Parte 2: la pérdida del padre menos la de los hijos. Con $\lambda = 0$ y $\gamma = 0$ es (la mitad de) la reducción de la suma de errores cuadráticos, el mismo criterio de un árbol de regresión. El término γ hace que una división solo valga la pena si reduce la pérdida al menos en γ. XGBoost hace crecer los árboles hasta una profundidad máxima y luego los poda, eliminando las divisiones con ganancia negativa.

2. Segundo orden. Para una pérdida cualquiera, XGBoost no usa solo el gradiente $g_i = \partial \ell / \partial \hat{y}_i$, sino también la segunda derivada $h_i = \partial^2 \ell / \partial \hat{y}_i^2$ (es un paso del método de Newton, en lugar de descenso por gradiente). Las fórmulas son las mismas de arriba, cambiando $\sum r_i$ por $-\sum g_i$ y el número de ejemplos n_j por $\sum h_i$:

$$w_j = -\frac{\sum_{i \in R_j} g_i}{\sum_{i \in R_j} h_i + \lambda}.$$

Con pérdida cuadrática, $g_i = -r_i$ y $h_i = 1$, y recuperamos exactamente lo anterior. Con la pérdida logística, $h_i = \sigma(\hat{y}_i)(1 - \sigma(\hat{y}_i))$, que es pequeño cuando el modelo ya está muy seguro.

3. Ideas de random forests. Al igual que un random forest, XGBoost puede entrenar cada árbol con una fracción de los ejemplos (submuestreo de filas) y considerar solo una fracción de las features en cada árbol o en cada división (submuestreo de columnas). Esto reduce la correlación entre los árboles y el overfitting.

Además de esto, XGBoost tiene muchas optimizaciones de ingeniería (paralelismo, manejo de valores faltantes, búsqueda aproximada de umbrales) que lo hacen muy rápido, pero la parte de aprendizaje automático es la que acabamos de ver.

En la librería `xgboost`, las clases `XGBRegressor` y `XGBClassifier` tienen la misma interfaz que scikit-learn. Los parámetros principales son `n_estimators` (B), `learning_rate` (ν), `max_depth`, `reg_lambda` (λ), `gamma` (γ), `subsample` (fracción de ejemplos por árbol) y `colsample_bytree` (fracción de features por árbol).

6. Resumen

- Un árbol de decisión divide el espacio de las features en regiones rectangulares haciendo preguntas de la forma $x_j < t$, de forma codiciosa y recursiva. Cada hoja predice la clase mayoritaria (clasificación) o el promedio de las etiquetas (regresión) de su región.
- En cada nodo se escoge la pregunta con mayor ganancia: la impureza del padre menos el promedio ponderado de la impureza de los hijos. El error de clasificación no sirve como impureza porque no es estrictamente cóncavo. La impureza de Gini, $\sum_c \hat{p}_c(1 - \hat{p}_c)$, es la probabilidad de equivocarse al etiquetar un ejemplo al azar con las proporciones de la región, y es estrictamente cóncava. La entropía cruzada se comporta casi igual.
- En regresión, la impureza es el error cuadrático medio. En clasificación binaria, Gini es el doble de la varianza de las etiquetas 0/1.
- Los árboles son interpretables, manejan variables categóricas y no necesitan estandarizar, pero tienen alta varianza. Se regularizan con reglas de parada y con poda sobre un conjunto de validación.
- El promedio de B modelos con varianza σ^2 y correlación ρ tiene varianza $\rho\sigma^2 + (1 - \rho)\sigma^2/B$. Bagging entrena cada modelo con una muestra bootstrap y reduce la varianza. Random forests además sortea las features disponibles en cada división, para reducir ρ. En ambos, aumentar B no produce overfitting.
- Boosting entrena modelos débiles de forma secuencial, cada uno enfocado en los errores de los anteriores, y reduce el sesgo. AdaBoost aumenta el peso de los ejemplos mal clasificados. Gradient boosting entrena cada árbol con los residuos (o, en general, con el gradiente negativo de la pérdida). En boosting, aumentar B sí puede producir overfitting.
- XGBoost es gradient boosting con regularización ($\gamma J + \frac{1}{2}\lambda \sum w_j^2$), que encoge los valores de las hojas hacia cero ($w_j = \sum r_i / (n_j + \lambda)$), aproximación de segundo orden y submuestreo de ejemplos y features.